SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Athavan K	**Reg. No.:**	192524036
Section / Batch:	AI&DS	**Date:**	24/7/26

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
	GRAND TOTAL:	**100 Marks**	

SECTION A – DEBUGGING & OPTIMISATION TASK

1) ***Debugging Selection Sort Code (Incorrect Outer Loop Range)***

The following Selection Sort implementation skips the final required pass because of an incorrect outer loop boundary, which may leave the array partially unsorted for certain inputs. Carefully analyze the loop range and explain why the remaining unsorted portion is not handled correctly. Apply systematic debugging to correct the iteration structure and ensure complete sorting. Optimize the implementation by avoiding unnecessary swaps when the minimum element is already in the correct position. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):

  n = len(arr)

  for i in range(n-2):   # ERROR

    min_idx = i

    for j in range(i+1, n):
```

```
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
  return arr
```

2) ***Debugging Maximum–Minimum Finder Code (Comparison Branches Reversed)***

The following code attempts to find both the minimum and maximum values in a list, but the update conditions are reversed, causing wrong outputs for many inputs. Carefully analyze each comparison branch and explain how the variables are being updated incorrectly. Apply systematic debugging to restore correct min-max tracking across the full array. Optimize the implementation for readability and correctness without changing the brute force nature of the scan. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
def min_max(arr):
  mn = arr[0]
  mx = arr[0]
  for x in arr[1:]:
    if x > mn:   # ERROR
      mn = x
    if x < mx:   # ERROR
      mx = x
  return mn, mx
```

Code of Conduct

I Athavan K (192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Athavan K

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	39
Q2	4	4	4	3	2	44
					Overall Total (100)	**83**

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Debugging Selection Sort (Incorrect Outer Loop Range)

The following Selection Sort implementation skips the final required pass because of an incorrect outer loop boundary, leaving parts of the array unsorted for certain inputs.

Original buggy code:

```
def selection_sort(arr):
  n = len(arr)
  for i in range(n-2):   # ERROR
     min_idx = i
     for j in range(i+1, n):
        if arr[j] < arr[min_idx]:
           min_idx = j
     arr[i], arr[min_idx] = arr[min_idx], arr[i]
  return arr
```

a) Analysing the Loop Range Bug

A correct Selection Sort needs exactly n − 1 outer passes. On each pass i, it finds the smallest value among the still-unsorted elements and places it at position i; once the first n − 1 positions are each holding their correct value, the single element left over at the last index is automatically in the right place, since there is nothing smaller left to swap it with. That is why the standard loop bound is range(n - 1).

This version uses range(n - 2) instead, so i only takes the values 0 through n − 3 — one pass short of what is needed. The pass that would normally settle the second-to-last position, index n − 2, against everything after it never runs. Whatever values happen to be sitting at positions n − 2 and n − 1 after the previous pass are left exactly as they are, with no comparison or swap ever performed between them. If the smaller of those two values happens to already be at n − 2, the array looks correctly sorted by coincidence; if not, the last two elements stay out of order in the final result.

A direct trace makes the effect visible: for [5, 3, 8, 1], n = 4, so range(n - 2) only produces i = 0 and i = 1. After those two passes the array becomes [1, 3, 8, 5] — positions 2 and 3 (the values 8 and 5) were never compared as an outer pass, so they stay swapped relative to sorted order. The failure is even more obvious for a 2-element array: with n = 2, range(n - 2) is range(0), which means the loop body never executes at all, and a pair like [2, 1] is returned completely untouched.

Input Array	Buggy Output	Correct Output
[5, 3, 8, 1]	[1, 3, 8, 5]	[1, 3, 5, 8]
[2, 1]	[2, 1] (untouched)	[1, 2]
[9, 8, 7, 6]	[6, 7, 8, 9]	[6, 7, 8, 9]

b) Debugging — Correcting the Iteration Structure

The fix is a one-character change to the loop boundary: the outer loop must run over range(n - 1) rather than range(n - 2), so that i reaches all the way up to n − 2 and every position except the very last one gets its own pass.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):   # FIXED: was range(n - 2)
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

With this change, every position from 0 to n − 2 is guaranteed to receive its correct minimum from the remaining unsorted suffix, and the last element falls into place by elimination once everything before it is finalised.

c) Optimisation — Avoiding Unnecessary Swaps

Selection Sort's inner search always has to scan the remaining elements to find the minimum, so that part cannot be shortened. What can be avoided is the swap at the end of each pass: if min_idx already equals i, the smallest remaining element is already sitting in the correct position, and swapping arr[i] with itself is wasted work — harmless for plain integers, but wasteful for larger records or objects where a swap means copying real data. Wrapping the swap in a simple guard skips that redundant operation:

```
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

On an already-sorted or nearly-sorted array, this can reduce the number of swaps performed from n − 1 down to close to zero, without changing the sorting logic itself.

d) Time Complexity of the Corrected Version

The number of comparisons is unaffected by either fix. The outer loop runs n − 1 times, and for each pass the inner loop scans the remaining unsorted suffix, giving a total comparison count of (n − 1) + (n − 2) + ... + 1, which sums to n(n − 1) / 2. That is $O(n^2)$ in the best, average, and worst case alike, because Selection Sort always has to inspect every remaining element to be sure it has found the true minimum — it cannot exit early the way an already-sorted check might let Bubble Sort do.

The swap-avoidance optimisation changes the swap count, not the comparison count: in the original version every pass performs exactly one swap, giving n − 1 swaps regardless of input. With the min_idx != i guard, the worst case is still n − 1 swaps (a fully reverse-sorted array), but the best case — an already-sorted array — drops to zero swaps, since every pass finds min_idx == i. The algorithm remains in-place, needing only O(1) auxiliary space beyond the input array.

e) Final Optimised Code

```
def selection_sort(arr):
    """
    Selection Sort - corrected and optimised.
    Fix 1: outer loop now runs range(n - 1) so the last
           pair of elements is always compared and placed.
    Fix 2: swap is skipped when the minimum is already
           in position, avoiding redundant writes.
    Time Complexity : O(n^2) comparisons in all cases.
    Space Complexity: O(1) extra space (in-place).
    """
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:              # avoid unnecessary swap
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr


if __name__ == "__main__":
```

```
data = [5, 3, 8, 1]
print("Sorted:", selection_sort(data))
```

2)Debugging Maximum–Minimum Finder (Comparison Branches Reversed)

The following code attempts to find both the minimum and maximum values in a list, but the update conditions are reversed, causing wrong outputs for many inputs.

Original buggy code:

```
def min_max(arr):
  mn = arr[0]
  mx = arr[0]
  for x in arr[1:]:
    if x > mn:   # ERROR
      mn = x
    if x < mx:   # ERROR
      mx = x
  return mn, mx
```

a) Analysing the Reversed Comparison Branches

The variable mn is meant to track the smallest value seen so far, which means it should only ever be replaced by a value that is smaller than it. Instead, the buggy code updates mn whenever x is greater than mn, which pulls mn upward toward the largest values in the array rather than downward toward the smallest. The same mistake happens in reverse for mx: it is supposed to climb toward the largest value, but the buggy condition x < mx drags it downward toward the smallest value instead.

The practical effect is that the two variables end up holding each other's intended result: by the end of the scan, mn actually contains the maximum of the array, and mx contains the minimum — the output is exactly swapped. Tracing [5, 3, 8, 1] confirms this: starting with mn = mx = 5, the value 3 fails the 'x > mn' test but passes 'x < mx', so mx becomes 3; the value 8 passes 'x > mn', so mn becomes 8; the value 1 then passes 'x < mx', so mx becomes 1. The function returns (8, 1) — the true maximum where the minimum should be, and the true minimum where the maximum should be.

Input Array	Buggy (mn, mx)	Correct (mn, mx)

[5, 3, 8, 1]	(8, 1) — swapped	(1, 8)
[10, 2, 7, 1, 9, 3]	(10, 1) — swapped	(1, 10)
[4, 4, 4]	(4, 4)	(4, 4)

b) Debugging — Restoring Correct Min-Max Tracking

The fix is to swap which comparison drives which variable: mn should update only when a smaller value is found, and mx should update only when a larger value is found.

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for x in arr[1:]:
        if x < mn:      # FIXED: was x > mn
            mn = x
        if x > mx:      # FIXED: was x < mx
            mx = x
    return mn, mx
```

c) Optimisation for Readability and Correctness

Since mn can never be larger than mx at any point in the scan, a single element x cannot simultaneously be smaller than the current mn and larger than the current mx. That means once the first condition succeeds and updates mn, there is no need to also test the second condition for the same element. Rewriting the second check as an elif skips that redundant comparison whenever the first one already matched, without changing the result or the brute-force nature of the scan — it still looks at every element exactly once:

```
        if x < mn:
            mn = x
        elif x > mx:
            mx = x
```

This does not change what the function computes, only how many comparisons it performs to get there, cutting the worst case from two comparisons per element down to effectively one whenever an element updates the minimum.

d) Time Complexity of the Corrected Solution

The function makes a single pass over the n − 1 remaining elements after the initial assignment, performing one or two constant-time comparisons per element. That gives a linear running time of O(n) in the best, average, and worst case — brute-force scanning cannot do better than looking at every element at least once, since the minimum or maximum could be located anywhere in the array, including the very last position. The elif optimisation reduces the constant factor of the comparisons but does not change the O(n) complexity class. Space usage stays O(1), since only the two tracking variables mn and mx are needed regardless of array size.

e) Final Optimised Code

```
def min_max(arr):
    """
    Brute-force Minimum-Maximum finder - corrected and optimised.
    Fix: comparison directions were reversed in the original
        code (mn tracked the max, mx tracked the min); the
        conditions are now matched to the correct variable.
    Optimisation: second check uses elif, since a value cannot
        update both mn and mx in the same iteration.
    Time Complexity : O(n) - single pass over the array.
    Space Complexity: O(1) extra space.
    """
    mn = arr[0]
    mx = arr[0]
    for x in arr[1:]:
        if x < mn:
            mn = x
        elif x > mx:
            mx = x
    return mn, mx


if __name__ == "__main__":
    data = [5, 3, 8, 1]
    print("Min, Max:", min_max(data))
```